



Municipalidad de San Isidro
Gerencia de Tecnología de Información y Comunicación

**Manual de
Estándares de Base de Datos
Versión <3.1>**

 Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación		Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing. Arturo Rodríguez

Historial de Revisiones

Fecha	Versión	Descripción	Autor
19/07/2007	0.1	Versión preliminar	Luis Ponce de León
06/08/2007	0.2	Modificación a la Definición de Tablas	Luis Ponce de León
05/07/2010	0.3	Modificaciones a la nomenclatura de campos que forman parte de la llave primaria de las Tablas.	Jimmy Rojas Pedro Campos
08/07/2010	0.4	Modificación a la Nomenclatura de Objetos de Base de Datos y Variables dentro de Packages, Functions y Stored Procedures.	Jimmy Rojas Pedro Campos
12/12/2013	1.0	Estándar de Tablespace	Luis Ponce de León
05/04/2018	2.0	Trigger, Sinonimos, Vistas y uso de BIND VARIABLES Y BULK COLLECT	Ing. Luis Ponce de León Sr. Johan Merello
14/09/2018	2.1	Tablas Históricas	Ing. Luis Ponce de León Sr. Johan Merello
01/09/2023	3.0	Revisión de estándares	Ing. Luis Ponce de León
18/09/2023	3.1	Revisión de estándares	Sr. Marco Vicente Castillo

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

Tabla de Contenidos

1. Introducción.....	4
1.1 <i>Propósito</i>	<i>4</i>
1.2 <i>Alcance</i>	<i>4</i>
1.3 <i>Resumen.....</i>	<i>4</i>
2. Estándar de nomenclatura de objetos de base de datos	5
2.1 <i>Esquemas y sistemas.....</i>	<i>5</i>
2.2 <i>Tablas</i>	<i>5</i>
2.3 <i>Columnas.....</i>	<i>8</i>
2.4 <i>Restricciones (Constraint)</i>	<i>9</i>
2.5 <i>Indices.....</i>	<i>11</i>
2.6 <i>Tablespace</i>	<i>12</i>
2.7 <i>Secuencias</i>	<i>12</i>
2.8 <i>Vistas</i>	<i>13</i>
2.9 <i>Paquetes.....</i>	<i>14</i>
2.10 <i>Procedimientos</i>	<i>14</i>
2.11 <i>Funciones.....</i>	<i>17</i>
2.12 <i>Disparadores</i>	<i>20</i>
2.13 <i>SINONIMOS</i>	<i>21</i>
2.14 <i>BIND VARIABLES Y BULK COLLECT</i>	<i>21</i>
3. Estándar de Nomenclatura de programación	24
3.1 <i>Nomenclatura de Variables</i>	<i>24</i>
3.2 <i>Definición de variables.....</i>	<i>25</i>
3.3 <i>Comentarios.....</i>	<i>25</i>
3.4 <i>Sangría.....</i>	<i>26</i>
3.5 <i>Anidamiento de estructuras</i>	<i>27</i>
3.6 <i>Comentarios.....</i>	<i>27</i>

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

Manual de estándares de Base de Datos

1. Introducción

El presente documento provee información a ser tomada en cuenta en el diseño físico e implementación de la base de datos de la Municipalidad de San Isidro.

La información expuesta en este documento define un conjunto de reglas de uso obligatorio por parte del equipo funcional de desarrollo de sistemas para la implementación de nuevos sistemas y/o mantenimiento de sistemas existentes.

1.1 Propósito

El propósito del presente estandar es proporcionar los lineamientos que uniformicen el diseño, implementación de la base de datos que soporta los sistemas de información para la Municipalidad de San Isidro.

Los usuarios del Documento son:

- El Administrador de Base de Datos que lo utiliza para asegurarse que la implementación de la base de datos es uniforme.
- Los miembros del equipo de desarrollo lo utilizan para implementar los procedimientos almacenados, funciones o paquetes.

1.2 Alcance

Proporcionar información para la creación de procedimientos, vistas, paquetes, funciones y otros objetos de oracle para los desarrolladores, para una mejor comprensión y lectura de la información almacenada en la base de datos. Así mismo proporciona información para la creación de niveles de seguridad de acuerdo a la importancia de los objetos previamente estandarizados.

1.3 Resumen

Después de esta introducción, el resto del documento está organizado en las siguientes secciones:

Estándar de nomenclatura de objetos de base de datos — proporciona una descripción de la forma como se nombran cada uno de los objetos (tablas, índices, vistas, etc.) a crear en el servidor de base de datos

Estándar de programación pl/sql — Proporciona una descripción de la forma en que se debe escribir los bloques (funciones, procedimientos, paquetes, disparadores) de programas en el servidor de base de datos.

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

2. Estándar de nomenclatura de objetos de base de datos

2.1 Esquemas y sistemas

Los nombres de los esquemas de la base de datos deben tener en cuenta la siguiente estructura

MSI+SiglaDeSistema

Los esquemas para la implementación de la base de datos están organizados de acuerdo a los sistemas definidos en la arquitectura del sistema de gestión municipal y se encuentran implementados de la siguiente manera:

- Sistema de Gestión Municipal (MSISGM).
- Sistema de Gestión administrativa (MSISGA).
- Sistema de Gestión Tributaria (MSISGT).
- Sistema de Planificación y Desarrollo Urbano (MSIPDU).
- Sistema de Gestión de Servicios Municipales (MSISGSM).
- Sistema de Control Legal (MSISCL).
- Sistema de Gestión de Salud (MSISGS).

2.2 Tablas

2.2.1. Definición de nombres de tablas

Para la definición de una tabla se debe tener en cuenta las siguientes consideraciones:

- ✓ Los nombres de tabla no se deben repetir en la Base de Datos.
- ✓ Los 3 primeros caracteres deben indicar las siglas del sistema al que pertenece la tabla.
- ✓ Los nombres de las tablas deberán ser definidos en singular y en mayúsculas bajo la siguiente estructura.

<SiglaDeSistema>_<NombreFuncionalDeTabla>

Ejemplo:

SGT_DOCUMENTO

- ✓ Para el caso de tablas con nombres compuestos el nombre de la tabla será definido de la siguiente manera:

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

<SiglaDeSistema>_<Nombrefuncional1><Nombrefuncional2>

Ejemplo:

Tablas padre:

SGT_DOCUMENTO

SGT_AREA

Tabla con nombre compuesto:

SGT_DOCUMENTOAREA

- ✓ Para el caso de creación de tablas temporales se usará el prefijo TMP en lugar de las siglas de sistema

TMP_NombreFuncionalDeTabla

Ejemplo:

TMP_DOCUMENTO

- ✓ Para el caso de creación de tablas históricas se usará el prefijo HIS junto con las siglas de sistema

SIGLA_HIS_NombreFuncionalDeTablaHistórica

Ejemplo:

GRAL_HIS_PERSONA

- ✓ Para el caso de creación de tablas de uso general se utilizará el prefijo GRAL en lugar de las siglas del sistema

GRAL_<NombreFuncionalDeTabla>

Ejemplo:

GRAL_DOCUMENTO

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

- ✓ Tablas que manejan cabecera y detalle.

Con el fin de mantener un orden en la identificación de las tablas y poder realizar de una manera rápida y fácil la búsqueda y localización de las tablas, es necesario mantener el mismo nombre de la tabla y adicionarle un sufijo (ejemplo: “DET”) para los casos de cabecera detalle.

La estructura para la definición de una tabla es la siguiente:

<SiglaDeSistema>_<Nombrefuncional1>DET<AbreviaturaFuncional>

Ejemplo:

SGS_PEDIDOSH

SGS_PEDIDOSHDET

2.2.2 Sintaxis de creación

a) Tablas simples

```
CREATE TABLE <NOMBRE DE LA TABLA>
( CAMPO1 <TIPO DE CAMPO>,
  CAMPO2 <TIPO DE CAMPO>,
  .....
)
<PARTICIONES>
TABLESPACE <NOMBRE DEL TABLESPACE>
```

<PARTICIONES>

Para el caso de las tablas que tengan que utilizar particiones, su uso puede variar de acuerdo al tipo de partición utilizado

- Hash Partitioned Table

PARTITION BY HASH (campo)

PARTITION <numero de particiones>

STORE IN (part1, part2, part3)

- Particiones por nombre de tablespace

PARTITION BY LIST (CAMPO)

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing. Arturo Rodríguez

(PARTITION <Nombre_part1> VALUES (VALORES)

TABLESPACE part1,

PARTITION <Nombre_part2> VALUES (VALORES)

TABLESPACE part2)

- Particiones por rango

PARTITION BY RANGE (campo)

(PARTITION <Nombre_part1>

VALUES LESS THAN (CONDICION)

TABLESPACE part1,

PARTITION <Nombre_part2>

VALUES LESS THAN (CONDICION)

TABLESPACE part1, ...)

Tablas de objetos

CREATE TYPE <Nombre de la tabla objeto> AS OBJECT

(CAMPO1 TIPO_DATO ,

CAMPO2 TIPO_DATO ,

CAMPOn TIPO_DATO

)

CREATE TABLE <Nombre de la tabla>

(CAMPO1 TIPO_DATO ,

CAMPO2 <Nombre de tipo de objeto> ,

CAMPOn TIPO_DATO

)

2.3 Columnas

Los nombres de los atributos deberán ir acompañados de un identificador que permitirá agrupar y relacionar el tipo de atributo.

La estructura del nombre de atributo es la siguiente:

<Identificador><NombreColumna>

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing. Arturo Rodríguez

TIPO DE DATO	IDENTIFICADOR	DESCRIPCION
CARÁCTER	COD[nombretabla]	Código Varchar de 2,4,6,8,10,...
CARÁCTER	EST	Estado de 1 hasta 3 caracteres, para valores únicos
CARÁCTER	TXT	Textos Varchar de 15,30,45,50,80,240 caracteres
CARÁCTER	FLG	Flag 1 carácter
CARÁCTER	TIP	Tipo Varchar de 2,4,6,8
FECHA	FEC	Fecha Date
NUMERICO	PCT	Porcentaje Number(6,3) ej.: 60(60%), 12.38(12.38%)
NUMERICO	NUM	Numero Number con decimales
BLOB	BLB	

Ejemplo:

CODPERSONA	VARCHAR2(3)
NUMPESO	NUMBER(4)
FECEMISION	DATE
FLGENTREGADO	VARCHAR2(1)
TIPCONTRIB	VARCHAR2(2)

2.4 Restricciones (Constraint)

2.4.1. CONSTRAINT PRIMARY KEY (Clave Primaria)

La estructura para la definición de un constraint es la siguiente:

PK_NombreFuncionalDeTabla

teniendo en cuenta las siguientes consideraciones:

- ✓ Las tablas deberán iniciarse con el **prefijo “PK”**.
- ✓ El siguiente carácter deberá ser un guión “_” que se utilizará como separador entre las siglas del constraint y el nombre funcional.

Sintaxis de creación para primary key:

ALTER TABLE <NOMBRE DE LA TABLA>

ADD CONSTRAINT <NOMBRE DE LA PK>

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing. Arturo Rodriguez

PRIMARY KEY (Campo1, Campo2.....)

USING INDEX

TABLESPACE <NOMBRE DEL TABLESPACE INDICE>

Ejemplo :

PK_DOCTRAMITE

PK_DOCOFICINA

6.2. CONSTRAINT FOREIGN KEY (Clave foránea)

Permite relacionar la tabla padre con la o las tablas hijas.

Estructura:

FK_NombreFuncionalTablaHija_NombreFuncionalTablaPadre

En el caso de usar FOREIGN compuestos la sintaxis debe ser la siguiente

FK_NombreFuncionalTablaHija_NombreFuncionalTablaPadre_Funcionalidad

Sintaxis de creación para foreign key:

ALTER TABLE <NOMBRE DE LA TABLA>

ADD CONSTRAINT <NOMBRE DE LA FK>

FOREIGN KEY (Campo o campos de la tabla Hijo que se va relacionar)

REFERENCES <TABLA PADRE>(Campo o campos de la tabla padre que se va relacionar)

Ejemplo :

FK_DOCTRAMITE_CONTRIBUYENTE

FK_DOCOFICINA_TIPODOCUMENTO

6.3. CONSTRAINT CHECK

Estructura

CHK_NombreFuncionalTabla_NombreColumna

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

Ejemplo

CHK_DOCTRAMITE_NUMFOLIOS

CHK_DOCOFICINA_CODCONTRIB

Un campo puede tener de uno o mas check constraints, para este caso se debe seguir la siguiente sintaxis.

CHK_NombreFuncionalTabla_NombreColumna_Funcionalidad

Sintaxis de creación para Check:

ALTER TABLE <NOMBRE DE LA TABLA>

ADD CONSTRAINT <NOMBRE DEL CHECK>

CHECK (Condición);

2.5 Índices

Se debe tener en cuenta en la creación de índices, el no superar los 9 índices por tabla.

Estructura

IDX_NombreFuncionalTabla_NombreFuncionalIndice

Sintaxis de creación para índices:

CREATE INDEX <NOMBRE DEL INDICE>

ON <TABLA>

(campo1, campo2,, Campo N)

LOGGING

TABLESPACE <NOMBRE DEL TABLESPACE>

Ejemplo

IDX_DOCTRAMITE_CODCONTRIB

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

2.6 Tablespace

Los tablespace permiten almacenar información del negocio municipal. En ese sentido es vital su disponibilidad de crecimiento para evitar contención y/o problemas de almacenamiento.

Datafile inicial 1024Mb
Datafile bajo 5Gb
Datafile medio 8Gb
Datafile alto 12Gb

Standard para almacenamiento de datafile
Unidad /u03 para tablas
Unidad /u04 para índices

Sintaxis para TABLESPACE

[MSI][SiglaDeSistema]_TAB : Tablespace para tablas.

[MSI][SiglaDeSistema]_IDX : Tablespace para índices.

Para datafiles

[msi][SiglaDeSistema]_tab[01..n].dbf : Para tablas (En minúscula).

[msi][SiglaDeSistema]_idx[01..n].dbf : Para índices (En minúscula).

Los tablespace para la implementación de las base de datos están organizados de acuerdo a los sistemas definidos en la arquitectura del sistema de gestión municipal y se encuentran implementados de la siguiente manera:

- Sistema de Gestión administrativa (MSISGA_TAB y/o MSISGA_IDX).
- Sistema de Gestión Tributaria (MSISGT_TAB y/o MSISGT_IDX).

2.7 Secuencias

Estructura

SEQ_AbreviaturaTabla+Nombrefuncional

Sintaxis de creación para SEQUENCE:

CREATE SEQUENCE <NOMBRE DEL SEQUENCE>
MINVALUE <Valor Mínimo – Numérico>

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

MAXVALUE <Valor Máximo - Numérico>
START WITH <Valor de inicio – Numérico>
INCREMENT BY <Valor de incremento – Numérico>;

Ejemplo

SEQ_EXPENUMERO

2.8 Vistas

Estructura :

V<SiglaDeSistema>_<NombreFuncionalVista>

NOMBREFUNCIONALDEVISTA: Indica el nombre de la vista ha ser creada, debe ser un nombre de hasta 20 caracteres.

Sintaxis de creación para la vista:

CREATE OR REPLACE VIEW VASIS_DOC_VAR

(campo1,campo2,....,campoN)

AS

<Consulta>

Buenas prácticas para la creación y uso de vistas.

- 1) Verificar que las consultas de las vista se encuentren bien relacionadas.
- 2) No incluir funciones dentro de las uniones de la consulta. Ej (trim, decode, etc)
- 3) Utilizar los índices asociados a las tablas.
- 4) No crear consultas como campo dentro de la consulta principal de la vista.
- 5) Verificar que la data sea consistente (valores nulos) para evitar sentencias sql no adecuadas.

Ejemplo:

VSGT_DOCUMENTOS

VSGT_DEUDORES

NOTA:

El uso de vistas no está permitido para los nuevos sistemas y futuras actualizaciones de sistemas actuales.

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

2.9 Paquetes

Permite agrupar uno o mas procesos relacionados de acuerdo a su funcionalidad.

Los packages permite elaborar procesos transaccionales complejos anidando uno o mas procesos semejantes; así mismo el package maneja de cursores avanzados para elaborar consultas.

Estructura:

PKG_<SiglaDeSistema><DescripciónFuncional>

Ejemplo:

PKG_SGT_UTILITARIOS

PKG_SGT_CONSULTASGENERALES

Los procedimientos incluidos dentro de un package deberán mantener la siguiente estructura:

PRC_(NOMBRE DEL PROCEDURE)

PRC

Prefijo que indica que el procedimiento esta contenido dentro de un package.

(NOMBRE DEL PROCEDURE)

Indica el nombre del procedimiento

Ejemplo:

PRC_VERIFICAESTADO

PRC_LISTADOCUMENTOS

2.10 Procedimientos

Estructura :

PRC_DESCRIPCIONFUNCIONAL

Ejemplo:

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

PRC_ENVIOEMAIL

PRC_LEERTEXTO

Sintaxis

CREATE OR REPLACE PROCEDURE <Nombre del Procedimiento> (sección de variables)

AS

/* Sección de definición de variables */

<Variable1> tipo ; -- Breve descripción

<Variable2> tipo; -- Breve descripción

<VariableN> tipo; -- Breve descripción

/* Sección de Comentarios e información */

Un procedure deberá incluir de manera comentada lo siguiente :

Para la creación

/*

AUTOR Nombres del creador o Modificador del procedure

FECHA DE CREACION Fecha de creación

FECHA DE MODIFICACION Fecha de modificación

DESCRIPCION Descripción corta del procedure

*/

Para la modificación

/*

AUTOR Nombres del creador o Modificador del procedure

FECHA DE MODIFICACION Fecha de modificación

DESCRIPCION Descripción corta de la modificación

*/

/* Cuerpo del procedimiento */

BEGIN

/* Todo segmento de transacción y consulta debe esta enmarcada dentro de un begin – end*/

/* Descripción de la actividad que realiza la consulta */

BEGIN

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

```

SELECT <Campo1,
        Campo2,
        Campo n
FROM   <Nombre de la Tabla>
WHERE  <Sección de Condiciones>
GROUP BY <Sección de agrupación>
HAVING  <Sección de condiciones agrupadas> ;
EXCEPTION  -- Sección de excepciones
WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20001,'Mensaje de error..');
END; -- Fin del Select

```

/* Descripción de la actividad que realiza la Actualización */

```

BEGIN
        UPDATE <Nombre de la Tabla>
SET      <Campo1> = Valor ,
        <Campo2> = Valor ,
        <CampoN> = Valor
WHERE  <Sección de Condiciones> ;
EXCEPTION  -- Sección de excepciones
WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20002,'Mensaje de error..');
        /* Nótese que la sección de errores se encuentra enumerada y de
        forma correlativa */
END; -- Fin del Update

```

/* Descripción de la actividad que realiza el Insert */

```

BEGIN
        INSERT
        INTO   <Nombre de la Tabla>
        (Campo1 ,

```


	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

Campo2

CampoN)

VALUES

(Valor1 ,

Valor2 ,

ValorN) ;

EXCEPTION -- Sección de excepciones

WHEN OTHERS THEN

RAISE_APPLICATION_ERROR(-20003,'Mensaje de error..');

/* Nótese que la sección de errores se encuentra enumerada y de

forma correlativa */

END; -- Fin del Insert

/* Descripción de la actividad que realiza el Delete */

BEGIN

DELETE

FROM <Nombre de la Tabla>

WHERE <Sección de Condiciones>;

EXCEPTION -- Sección de excepciones

WHEN OTHERS THEN

RAISE_APPLICATION_ERROR(-20004,'Mensaje de error..');

/* Nótese que la sección de errores se encuentra enumerada y de

forma correlativa */

END; -- Fin del Delete

/* Sección de var

END; -- Fin del procedimiento

2.11 Funciones

Estructura :

FNC_(MOD)_DESCRIPCION

FNC

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing. Arturo Rodríguez

Prefijo que indica que es una función.

(MOD): Indica las iniciales del modulo al que pertenece.

Puede ser:

TDI: Tramite Documentario Interno

TDE: Tramite Documentario Externo

ECO: Ejecutoría Coactiva

DESCRIPCION: Indica una descripción de la función y hace referencia a la opción a trabajar.

Ejemplo:

FNC_TDE_IMPORTELETRAS

CREATE OR REPLACE FUNCTION <Nombre de la Función>

(sección de variables)

RETURN <tipo> IS

Una función deberá incluir de manera comentada lo siguiente :

/*

AUTOR Nombres del creador o Modificador de la función

FECHA DE CREACION Fecha de creación

FECHA DE MODIFICACION Fecha de modificación

DESCRIPCION Descripción corta de la función.

*/

BEGIN

/* Todo segmento de transacción y consulta debe estar enmarcada dentro de un begin – end*/

/* Descripción de la actividad que realiza la consulta */

BEGIN

SELECT <Campo1,

Campo2,

Campo n

FROM <Nombre de la Tabla>

WHERE <Sección de Condiciones>

GROUP BY <Sección de agrupación>

HAVING <Sección de condiciones agrupadas> ;

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

```

EXCEPTION  -- Sección de excepciones
WHEN OTHERS THEN
    RAISE_APPLICATION_ERROR(-20001,'Mensaje de error..');
END; -- Fin del Select

```

/* Descripción de la actividad que realiza la Actualización */

```

BEGIN
    UPDATE <Nombre de la Tabla>
SET      <Campo1> = Valor ,
        <Campo2> = Valor ,
        <CampoN> = Valor
WHERE <Sección de Condiciones> ;
EXCEPTION  -- Sección de excepciones
WHEN OTHERS THEN
    RAISE_APPLICATION_ERROR(-20002,'Mensaje de error..');
    /* Nótese que la sección de errores se encuentra enumerada y de
    forma correlativa */
END; -- Fin del Update

```

/* Descripción de la actividad que realiza el Insert */

```

BEGIN
    INSERT
INTO    <Nombre de la Tabla>
        (Campo1 ,
          Campo2
          CampoN)
VALUES
        ( Valor1 ,
          Valor2 ,
          ValorN) ;
EXCEPTION  -- Sección de excepciones

```

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

```

WHEN OTHERS THEN
    RAISE_APPLICATION_ERROR(-20003,'Mensaje de error..');

/* Nótese que la sección de errores se encuentra enumerada y de
forma correlativa */

END; -- Fin del Insert

/* Descripción de la actividad que realiza el Delete */
BEGIN
    DELETE
FROM    <Nombre de la Tabla>
WHERE   <Sección de Condiciones>;
EXCEPTION  -- Sección de excepciones
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20004,'Mensaje de error..');

/* Nótese que la sección de errores se encuentra enumerada y de
forma correlativa */

END; -- Fin del Delete

/* Sección de variables

RETURN(VARIABLE);

END;
```

2.12 Disparadores

Se utiliza en los siguientes casos

- En el uso de una integridad lógica entre tablas.
- Cuando la operación de un evento del trigger es constante y sin variación cada vez que se ejecute,

Estructura:

TR_(OP)NOMBREFUNCIONALTABLA

TR : Identifica que se trata de un triggers.

(OP) :

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

	BI	BEFORE INSERT
	BU	BEFORE UPDATE
	BD	BEFORE DELETE
AI	AFTER INSERT	
	AU	AFTER UPDATE
	AD	AFTER DELETE

NOMBREFUNCIONALTABLA Nombre de la tabla

Ejemplos:

TR_BIDOCTRAMITE
TR_AICONTRIBUYENTE

NOTA:

El uso de trigger no está permitido para los nuevos sistemas y futuras actualizaciones de sistemas actuales, debido a la congestión que ocasionan.

2.13 SINONIMOS

No están permitidos en la implementación de nuevos sistemas y futuras actualizaciones de sistemas actuales.

2.14 BIND VARIABLES Y BULK COLLECT

BULK COLLECT

El Bulk Collect permite administrar el uso de la memoria de las transacciones, llevando el proceso al cliente y permitiendo liberar la memoria utilizada en el servidor de base de datos. El tiempo de proceso es 10 veces menor con el Bulk Collect; es decir del 100% del tiempo convencional con el bulk collect tomaría el 9% del tiempo total convencional.

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing. Arturo Rodríguez

Ejemplo de uso:

EJEMPLO CON CURSOR, BULK COLLECT, FOR ALL, LIMIT 100 (Con Límite)
<pre> set serveroutput on; declare l_start number; cursor c_cursor is select transaccion,codigo_tributo from msirentas.detalle_de_transacciones; type tab_detalle_de_transacciones is table of c_cursor %rowtype; reg_detalle_de_transacciones tab_detalle_de_transacciones; begin l_start := dbms_utility.get_time; open c_cursor; loop fetch c_cursor bulk collect into reg_detalle_de_transacciones limit 100; forall i in 1..reg_detalle_de_transacciones.count insert into prueba.detalle_de_transacciones values reg_detalle_de_transacciones(i); commit; exit when c_cursor%NOTFOUND; end loop; close c_cursor; dbms_output.put_line('#SEG: (dbms_utility.get_time-l_start)/100); end; / </pre>
Explicación: En este ejemplo se utiliza la cláusula limit que permite paginar la memoria utilizada para el proceso. Es decir no consume de golpe todo el recurso de la PC.

BIND VARIABLES

Usando este método se puede lograr lo que se explicara a continuación:

Ejemplos de dos consultas con diferente dato:

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

```
select * from empleado where empno=1234
```

y

```
select * from empleado where empno=5678
```

Ejecutar estas dos sentencias implicara dos parseos diferentes y dos entradas diferentes en la [library cache](#). Sin embargo, estas sentencias hacen lo mismo; la única diferencia radica en el valor. Para evitar estas duplicidades, las aplicaciones deben utilizar algún mecanismo de variables que tome los valores reales en tiempo de ejecución. La sentencia SQL debería ser del tipo:

```
select * from emp where empno=:x
```

para que pueda ser ejecutada múltiples veces con diferentes valores para la variable :x. Una aplicación bien diseñada haría un solo parse de esta sentencia y repetiría la ejecución tantas veces como sea necesario con distintos valores para la variable de bind que utilice en el programa de la aplicación. Esta forma de programar no solo hace que un simple programa funcione mejor sino que también asegura la escalabilidad cuando se ejecuten múltiples instancias concurrentes del mismo programa compartiendo el mismo código SQL en la library cache. Esta ha sido siempre la recomendación de Oracle para la programación de aplicaciones.

Ejemplo de uso:

```
PROCEDURE PS_PROCESOSHR (
    pvcCODUSUARIOREG IN MSISAT.SAT_HR.CODUSUARIOREG%TYPE,
    pdtFECREG         IN MSISAT.SAT_HR.FECREG%TYPE,
    pcrPROCESOS       OUT CursorDatos)
IS
BEGIN
    /*SOLO FT y FA*/
    OPEN pcrPROCESOS FOR q'[SELECT
                                MIN(HR.FECREG) FECPROCESO,
                                COUNT(1) NUMANIOSAFECTADOS,
                                HR.CODCONTRIBUYENTE,
                                MIN(P.CODPERSONA) CODPERSONA,
                                MIN(P.NUMVERSIONPERSONA)
NUMVERSIONPERSONA,
                                MIN(P.TXTRAZONSOCIAL) TXTRAZONSOCIAL,
                                HR.CORRELATIVOANTIGUO
FROM SAT_HR HR
INNER JOIN SAT_CONTRIBUYENTE C ON
C.CODCONTRIBUYENTE = HR.CODCONTRIBUYENTE
INNER JOIN SAT_PERSONA P ON
C.CODPERSONA = P.CODPERSONA AND C.NUMVERSIONPERSONA = P.NUMVERSIONPERSONA
WHERE
                                TRUNC(HR.FECREG)=TRUNC(:FECREG) AND
                                HR.CODUSUARIOREG = :CODUSUARIOREG AND
```

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

```

HR.CODTIPODOCINGRESO = :CODTIPODOC
AND NVL(TXTENVIO, 'NO') = :VAR1
AND NVL(TXTVALIDA, 'NO') = :VAR2
GROUP BY HR.CORRELATIVOANTIGUO,
ORDER BY MIN(HR.FECREG) DESC] '
USING pdtFECREG,pvcCODUSUARIOREG,CODTIPODOC, VAR1, VAR2;
END PS_PROCESOSHR;
```

3. Estándar de Nomenclatura de programación

3.1 Nomenclatura de Variables

Estructura:

<TipoVariable><TipoDato>_<Nombrevariable>

TipoVariable: Define el alcance de la variable

Tipo	Descripción
P	Parametro
L	Local

TipoDato: Define el prefijo del tipo de dato de la variable

Prefijo	Descripción
VC	Varchar2
NM	Number
DT	Date
IN	Integer
CR	Cursor

Nombrevariable: Define el nombre funcional de la variable para lo cual se debe tener las siguientes

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing. Arturo Rodríguez

consideraciones:

- las variables jamás deben tener doble significado
- No utilizar la misma variable para almacenar datos diferentes

Ejemplos:

lvcCODOFICINA
ldtFECNACIMIENTO
linCUENTA

3.2 Definición de variables

<Variable> <tipo de dato>; -- descripción de variable

Ejemplo:

lvcCODPERSONA VARCHAR2(8); -- Código de persona
linCONTADORPISOS INT; -- Variable contador de sentencia repetitiva

3.3 Comentarios

Comentarios de bloque
/* comentario de bloque */

Comentarios de Línea
-- Comentario de línea

Uso obligatorio

- Inmediatamente después de la definición de un bloque pl/sql (función, procedimiento, disparador)

CREATE OR REPLACE FUNCTION

/*

Autor : Nombre del Autor
Fecha de creación : Fecha de creación del bloque pl/sql
Descripción : Descripción de la funcionalidad que implementa el bloque pl/sql
Argumentos :
Argumento1 : Descripción de argumento 1
Argumento2 : Descripción de argumento 2
...
ArgumentoN : Descripción de argumento N
Dependencias : Lista de funciones, procedimientos y/o paquetes a los que invoca
Modificaciones
Fecha : Fecha de modificación
Autor : Nombre del autor de la modificación
Descripción : Descripción de la modificación

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

*/

- En cada línea de definición de variables
Variable tipo de dato; -- comentario

lvcODPERSONA VARCHAR2(8); -- Código de persona

- Línea anterior al inicio de un bloque y en la línea de finalización del bloque
/* comentario */
BEGIN
END; -- comentario de finalización de bloque
- En cada línea de definición y finalización de sentencias de control condicional o repetitivas
IF condición THEN -- comentario de inicio de sentencia de control de condición

END IF -- comentario de fin de control de condición

WHILE condicion LOOP -- comentario de inicio de sentencia repetitiva

END LOOP -- comentario de finalización de sentencia repetitiva

En el caso el comentario abarque mas de una línea, describir el comentario en la línea anterior al inicio de la sentencia o en la línea posterior al fin de sentencia

- Antes de cada sentencia DML, DDL o DCL

/* comentario */
Sentencia SQL

Ejemplo

/* recupera información de contribuyentes morosos */
SELECT codcontribuyente, f_moras(codcontribuyente, anio) as saldo
FROM contribuyente

- asdas

3.4 Sangría

Las sangrías en bloques pl/sql deben ser de tres espacios en blanco y deben ser hecho con la barra espaciadora. No se debe utilizar la tecla TAB

Ejemplo

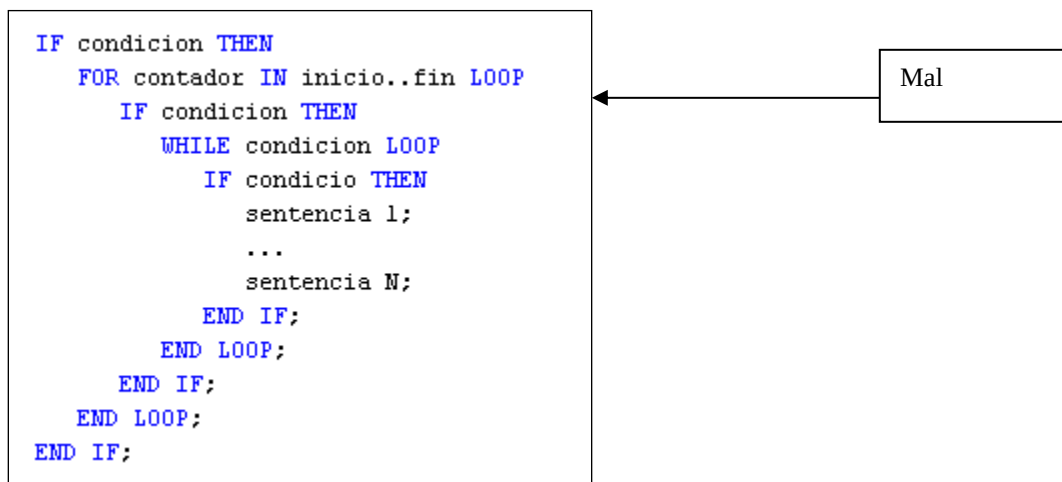
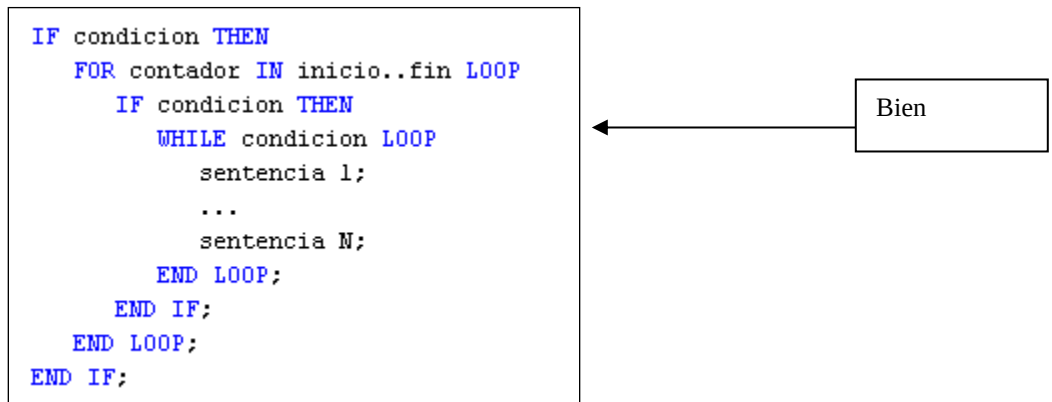
```
BEGIN
  VI_CONTADOR := 0;
  IF condicion THEN
    VD_FECHA := SYSDATE;
  END IF;
END;
```

Sangría

	Municipalidad de San Isidro Gerencia de Tecnología de Información y Comunicación	Fecha de creación: 19/07/2007 Fecha de revisión: 18/09/2023
Operación: Manual de Estándares Base de Datos		Versión: 3.1
Elaborado por: Ing. Luis Timir Ponce de Leon A.	Revisado por: Ing. Wilber Peña, Sr. Marco Vicente Castillo	Aprobado por: Ing .Arturo Rodriguez

3.5 Anidamiento de estructuras

Todo bloque pl/sql a implementar debe tener a lo mucho 4 estructuras de control anidadas, si surgen mas de 4 estructuras anidadas debe aplanarse factorizando el código que permita cumplir con esta regla.



3.6 Comentarios

- Norma Técnica Peruana NTP ISO/IEC 12207 publicada con Resolución Ministerial N° 179-2004-PCM